Mathematics and Algorithms Behind Vector Database Models

April 3, 2026

1 Introduction and Motivation

The "curse of dimensionality" makes exact nearest-neighbour search intractable in high-dimensional spaces: given n points in $\mathbb{R}^d$ and a query $q \in \mathbb{R}^d$, naïve linear scan requires $\mathcal{O}(nd)$ time. When n is in the hundreds of millions and d ranges from 128 to 1536 (as is common with modern neural embeddings), this cost is prohibitive.

Definition 1.1 (Nearest Neighbour Search — NNS)**.** Given a dataset $X = \{x_1, x_2, \ldots, x_n\} \subset \mathbb{R}^d$, a distance function $\delta : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}_{\geq 0}$, and a query point $q \in \mathbb{R}^d$, the *nearest neighbour* of q in X is

$$\mathrm{NN}(q) = \arg\min_{x \in X} \ \delta(q, x).$$

The k-nearest neighbour variant (k-NN) returns the k points with smallest distance.

Definition 1.2 (c-Approximate Nearest Neighbour — c-ANN)**.** For $c \geq 1$, a data structure solves c-ANN if, for any query q, it returns a point x^* such that

$$\delta(q, x^*) \ \leq \ c \cdot \delta\big(q, \mathrm{NN}(q)\big).$$

Vector databases implement c-ANN (often with c very close to 1) by combining clever indexing with optional lossy compression. The remainder of this document is organised as follows: Section 2 formalises distance metrics; Section 3 covers brute-force search; Sections 4–11 treat progressively more sophisticated algorithms; and Section 13 discusses practical system design.

2 Distance and Similarity Metrics

The choice of metric δ profoundly impacts both correctness and efficiency.

2.1 Euclidean (L^2) Distance

Definition 2.1. For $u, v \in \mathbb{R}^d$,

$$\delta_{L^2}(u, v) = \|u - v\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}.$$

Properties: a true metric (non-negativity, identity of indiscernibles, symmetry, triangle inequality). Squared Euclidean distance $\delta_{L^2}^2$ is often used to avoid the square root while preserving ranking.

2.2 Cosine Similarity and Angular Distance

Definition 2.2.

$$\mathrm{sim}_{\cos}(u,v) = \frac{\langle u, v\rangle}{\|u\|_2 \; \|v\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_i u_i^2}\,\sqrt{\sum_i v_i^2}}.$$

The corresponding *cosine distance* is $\delta_{\cos}(u,v) = 1 - \mathrm{sim}_{\cos}(u,v)$. Note that cosine distance is *not* a metric in the strict sense (it violates the triangle inequality), but the *angular distance*

$$\delta_{\mathrm{ang}}(u,v) = \frac{1}{\pi}\arccos\big(\mathrm{sim}_{\cos}(u,v)\big)$$

is a valid metric on the unit sphere.

Remark 2.1. When all vectors are L^2-normalised, Euclidean distance and cosine distance are monotonically related:

$$\|u-v\|_2^2 = 2\big(1 - \mathrm{sim}_{\cos}(u,v)\big) = 2\,\delta_{\cos}(u,v).$$

This identity is exploited in many systems to reduce cosine search to Euclidean search after a normalisation pre-processing step.

2.3 Inner Product (Maximum Inner Product Search — MIPS)

Definition 2.3.

$$\mathrm{MIPS}(q) = \arg\max_{x\in X} \; \langle q, x\rangle\,.$$

MIPS is not directly a metric search problem. A well-known reduction transforms MIPS into NNS by appending an extra dimension:

$$\tilde{x} = \big(x;\; \sqrt{1 - \|x\|_2^2}\big), \qquad \tilde{q} = (q;\; 0).$$

Then $\|\tilde{q} - \tilde{x}\|_2^2 = \|q\|_2^2 - 2\,\langle q, x\rangle + 1$, so minimising Euclidean distance on the augmented vectors is equivalent to maximising the inner product.

2.4 Manhattan (L^1) and Chebyshev (L^∞) Distances

$$\delta_{L^1}(u,v) = \sum_{i=1}^{d} |u_i - v_i|, \tag{1}$$

$$\delta_{L^\infty}(u,v) = \max_{1\le i\le d} |u_i - v_i|. \tag{2}$$

These are less common in embedding search but appear in specialised domains (e.g., L^1 in sparse feature spaces).

2.5 Hamming Distance

Definition 2.4. For binary vectors $u, v \in \{0,1\}^d$,

$$\delta_H(u,v) = \sum_{i=1}^{d} \mathbf{1}[u_i \neq v_i] = \mathrm{popcount}(u \oplus v).$$

Hamming distance is critical for binary hash codes produced by LSH and binary quantisation schemes. Modern CPUs compute `POPCNT` in a single instruction, making Hamming comparisons extremely fast.

3 Flat (Brute-Force) Search

3.1 Algorithm

The simplest approach scans all n vectors and maintains a max-heap of size k.

Algorithm 1 Flat k-NN Search

Require: Dataset $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, query $q \in \mathbb{R}^d$, integer k
Ensure: k nearest neighbours of q

```
H ← empty max-heap of capacity k                    ▷ keyed by distance
for i = 1 to n do
    d_i ← δ(q, x_i)
    if |H| < k then
        H.push(x_i, d_i)
    else if d_i < H.top().key then
        H.pop();  H.push(x_i, d_i)
    end if
end for
return H
```

3.2 Complexity

- **Time:** $\mathcal{O}(nd + n \log k)$ per query.
- **Space:** $\mathcal{O}(nd)$ for storage.
- **Preprocessing:** $\mathcal{O}(1)$ (no index built).

Although asymptotically poor, flat search provides *perfect recall* and is surprisingly competitive for small datasets ($n < 10^4$), especially with SIMD-accelerated distance computation.

4 Tree-Based Methods

4.1 k-d Trees

Definition 4.1 (k-d Tree [23]). A k-d tree is a binary space-partitioning tree in which each internal node stores a splitting hyperplane orthogonal to one of the d coordinate axes. At depth ℓ, the splitting dimension is $\ell \bmod d$.

Algorithm 2 k-d Tree Construction

```
function BUILD(P, depth)
    if |P| = 0 then return NIL
    end if
    axis ← depth mod d
    Sort P by coordinate axis
    m ← ⌊|P|/2⌋                                     ▷ median index
    node.point ← P[m]
    node.left ← BUILD(P[0..m−1], depth + 1)
    node.right ← BUILD(P[m+1..|P|−1], depth + 1)
    return node
end function
```

Query. Nearest-neighbour search descends the tree, pruning sub-trees whose bounding boxes are farther than the current best candidate.

Complexity.

- Build: $\mathcal{O}(n \log n)$.
- Query (best case): $\mathcal{O}(\log n)$ for low d.
- Query (worst case): $\mathcal{O}(n^{1-1/d})$ — exponential degradation in d.

Due to the curse of dimensionality, k-d trees are effective only for $d \lesssim 20$.

4.2 Ball Trees

Ball trees use hyper-spheres instead of axis-aligned hyperplanes.

Definition 4.2 (Ball Tree [24])**.** Each internal node v stores a centre $c_v \in \mathbb{R}^d$ and a radius $r_v \geq 0$ such that all descendant points lie within the ball $B(c_v, r_v)$. The children partition the points into two clusters.

Pruning rule: a ball $B(c_v, r_v)$ can be pruned if $\delta(q, c_v) - r_v > d_{\text{best}}$, where d_{best} is the current best distance.

Ball trees handle general metrics (not just L^2) and are more resilient to moderate dimensionality than k-d trees, but still suffer at very high d.

4.3 Randomised Partition Trees (Annoy)

The *Approximate Nearest Neighbours Oh Yeah* (Annoy) library [12] builds a forest of randomised binary trees [22]. Each split is defined by a random hyperplane passing through the midpoint of two randomly selected data points.

Definition 4.3 (Random Projection Split)**.** Given two points $a, b \in \mathbb{R}^d$, the splitting hyperplane has normal $w = a - b$ and passes through the midpoint $\frac{a+b}{2}$. A point x goes to the left child if $\left\langle w, x - \frac{a+b}{2} \right\rangle \leq 0$.

At query time, Annoy explores multiple trees simultaneously using a priority queue, merging candidate sets. Building T trees on n points costs $\mathcal{O}(Tn \log n)$; querying has empirical complexity $\mathcal{O}(T\, d\, \log n)$ per query.

5 Locality-Sensitive Hashing (LSH)

5.1 Formal Framework

The LSH framework was introduced by Indyk and Motwani [2] and later refined by Datar et al. [6].

Definition 5.1 ((r, cr, p_1, p_2)-Sensitive Hash Family [2])**.** A family $\mathcal{H}$ of functions $h : \mathbb{R}^d \to \mathcal{U}$ is (r, cr, p_1, p_2)-sensitive for distance δ if for any $u, v \in \mathbb{R}^d$:

$$\delta(u, v) \leq r \implies \Pr[h(u) = h(v)] \geq p_1, \tag{3}$$

$$\delta(u, v) \geq cr \implies \Pr[h(u) = h(v)] \leq p_2, \tag{4}$$

with $p_1 > p_2$ and $c > 1$.

The *LSH exponent* $\rho = \frac{\ln p_1}{\ln p_2} \in (0, 1)$ governs the space–time trade-off.

5.2 LSH for Angular Distance (SimHash / Hyperplane LSH)

SimHash was introduced by Charikar [3].

Definition 5.2 (SimHash [3])**.** Draw $r \sim \mathcal{N}(0, I_d)$. Define

$$h_r(x) = \operatorname{sign}\big(\langle r, x\rangle\big) \in \{+1, -1\}.$$

Then $\Pr[h_r(u) = h_r(v)] = 1 - \frac{\theta(u,v)}{\pi}$, where $\theta(u, v) = \arccos(\mathrm{sim}_{\cos}(u, v))$.

By concatenating K independent hash bits to form a *signature* and using L independent hash tables, the query procedure becomes:

Algorithm 3 LSH Query (L tables, K bits each)

Require: Query q, hash functions $\{h_{l,1}, \ldots, h_{l,K}\}_{l=1}^{L}$, hash tables $\{T_l\}_{l=1}^{L}$

```
candidates ← ∅
for l = 1 to L do
    g ← (h_{l,1}(q), ..., h_{l,K}(q))          ▷ K-bit signature
    candidates ← candidates ∪ T_l[g]
end for
return k-NN(q, candidates) via brute-force
```

5.3 LSH for Euclidean Distance (E2LSH)

Definition 5.3 (E2LSH Hash [6])**.** Let $a \sim \mathcal{N}(0, I_d)$ and $b \sim \mathrm{Uniform}[0, w)$ for bucket width $w > 0$. Then

$$h_{a,b}(x) = \left\lfloor \frac{\langle a, x\rangle + b}{w} \right\rfloor.$$

The collision probability depends on $\delta_{L^2}(u, v)/w$ and has a closed-form expression involving the Gaussian CDF.

5.4 Complexity

Theorem 5.1. For an (r, cr, p_1, p_2)-sensitive family with $\rho = \frac{\ln p_1}{\ln p_2}$, there exists an LSH data structure solving c-ANN with:

- Space: $\mathcal{O}(n^{1+\rho} + nd)$.
- Query time: $\mathcal{O}(n^{\rho}\, d)$.
- Preprocessing: $\mathcal{O}(n^{1+\rho}\, d)$.

For SimHash on angular distance with approximation ratio c, $\rho = 1/c$.

5.5 Multi-Probe LSH

Standard LSH requires many tables. Multi-probe LSH [7] reduces this by probing *neighbouring* buckets—those obtained by flipping one or more hash bits—using a perturbation scheme ranked by expected distance. This can reduce L by an order of magnitude while maintaining recall.

6 Quantisation Methods

6.1 Scalar Quantisation (SQ)

Definition 6.1 (Scalar Quantisation)**.** Each floating-point coordinate $x_i \in \mathbb{R}$ is independently mapped to one of 2^b levels. For uniform quantisation with b bits:

$$\hat{x}_i = \mathrm{round}\left(\frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \cdot (2^b - 1)\right).$$

Common choices are $b \in \{4, 8\}$. With $b = 8$, storage shrinks from $4d$ bytes (float32) to d bytes per vector. Distance computation uses look-up tables or integer SIMD.

6.2 Product Quantisation (PQ)

Product quantisation, introduced by Jégou et al. [9], is the workhorse compression scheme for large-scale vector search.

Definition 6.2 (Product Quantisation [9])**.** Decompose $\mathbb{R}^d$ into M disjoint sub-spaces of dimension d/M each:

$$x = \left(x^{(1)}, x^{(2)}, \ldots, x^{(M)}\right), \qquad x^{(m)} \in \mathbb{R}^{d/M}.$$

In each sub-space, learn a codebook $C^{(m)} = \{c_1^{(m)}, \ldots, c_{K_s}^{(m)}\}$ via k-means (typically $K_s = 256$). A vector x is encoded as

$$\mathrm{PQ}(x) = \left(q^{(1)}(x), \ldots, q^{(M)}(x)\right), \quad q^{(m)}(x) = \arg\min_j \left\| x^{(m)} - c_j^{(m)} \right\|_2 .$$

Each code $q^{(m)}(x) \in \{1, \ldots, K_s\}$ requires $\lceil \log_2 K_s \rceil = 8$ bits for $K_s = 256$, so the total code is M bytes.

Asymmetric Distance Computation (ADC). At query time, we compute exact sub-space distances between q and each centroid, then approximate the full distance:

$$\hat{\delta}(q, x)^2 = \sum_{m=1}^{M} \left\| q^{(m)} - c_{q^{(m)}(x)}^{(m)} \right\|_2^2 .$$

The $M \times K_s$ distance table is precomputed in $\mathcal{O}(MK_s \cdot d/M) = \mathcal{O}(K_s d)$ time. Then each vector comparison costs only $\mathcal{O}(M)$ look-ups.

Algorithm 4 PQ Search with ADC

Require: Codebooks $\{C^{(m)}\}_{m=1}^{M}$, codes $\{\mathrm{PQ}(x_i)\}_{i=1}^{n}$, query q

```
for m = 1 to M do                                   ▷ Build distance table
    for j = 1 to K_s do
        D[m][j] ← ‖q^(m) − c_j^(m)‖_2^2
    end for
end for
for i = 1 to n do                                   ▷ Scan all codes
    d̂_i ← Σ_{m=1}^{M} D[m][PQ(x_i)_m]
end for
return top-k by smallest d̂_i
```

6.3 Optimised Product Quantisation (OPQ) [10]

Standard PQ assumes the sub-spaces are axis-aligned. OPQ [10] learns a rotation matrix $R \in \mathbb{R}^{d \times d}$ such that the rotated data Rx has minimised quantisation error when partitioned into sub-spaces:

$$R^* = \arg\min_{R \in O(d)} \sum_{i=1}^{n} \left\| Rx_i - \widetilde{Rx_i} \right\|_2^2 ,$$

where $\widetilde{Rx_i}$ is the PQ reconstruction of Rx_i. This is solved by alternating between codebook updates (standard k-means) and rotation updates (Procrustes).

6.4 Additive Quantisation (AQ) and Residual Quantisation (RQ)

Definition 6.3 (Additive Quantisation [20])**.** Represent each vector as a sum of codewords from M codebooks (which now span the *full* d-dimensional space):

$$\hat{x} = \sum_{m=1}^{M} c_{j_m}^{(m)}, \qquad c_{j_m}^{(m)} \in \mathbb{R}^d.$$

The codes $(j_1, \ldots, j_M)$ are chosen jointly to minimise $\|x - \hat{x}\|_2^2$.

This is more expressive than PQ (since codebooks interact across dimensions) but encoding is NP-hard; beam search is used in practice.

Definition 6.4 (Residual Quantisation [21])**.** A greedy special case of AQ: quantise x with the first codebook to get residual $r_1 = x - c_{j_1}^{(1)}$, then quantise r_1 with the second codebook, and so on:

$$r_0 = x, \qquad j_m = \arg\min_j \left\| r_{m-1} - c_j^{(m)} \right\|, \qquad r_m = r_{m-1} - c_{j_m}^{(m)}.$$

RQ often achieves better distortion than PQ at the same code length, especially when combined with a learnt rotation (RQ with OPQ).

6.5 Binary Quantisation

The simplest quantisation: $\hat{x}_i = \mathrm{sign}(x_i)$. Reduces d-dimensional float32 vectors to d-bit strings. Distances are approximated via Hamming distance, enabling extremely fast comparison ($\sim$1 clock cycle per 64 bits via `POPCNT`). However, the information loss is severe; binary quantisation is typically used only as a *re-ranking filter* [26]: retrieve a large candidate set via Hamming, then re-rank with exact distances.

7 Inverted File Index (IVF)

7.1 Core Idea

Partition the dataset into C Voronoi cells via k-means clustering with centroids $\mu_1, \ldots, \mu_C$. Each vector x is assigned to its closest centroid. At query time, only the n_{probe} closest cells are searched.

Definition 7.1 (IVF Assignment)**.**

$$\mathrm{cell}(x) = \arg\min_{c \in \{1,\ldots,C\}} \ \delta(x, \mu_c).$$

Algorithm 5 IVF Search

Require: Centroids $\{\mu_c\}_{c=1}^{C}$, inverted lists $\{L_c\}_{c=1}^{C}$, query q, n_{probe}

1: $S \leftarrow n_{\text{probe}}$ closest centroids to q
2: candidates $\leftarrow \bigcup_{c \in S} L_c$
3: **return** k-NN$(q, \text{candidates})$

7.2 IVF-PQ: Combining IVF with Product Quantisation

Store PQ codes instead of full vectors in each inverted list. The residual $r_x = x - \mu_{\mathrm{cell}(x)}$ is quantised, improving distortion. This is the backbone of the Faiss library's [14] most widely-used index (`IVF`C`,PQ`M).

Complexity.

- Index build: $\mathcal{O}(ndC_{\text{iter}})$ for k-means + $\mathcal{O}(nd)$ for PQ encoding.
- Query: $\mathcal{O}(Cd + n_{\text{probe}} \cdot \frac{n}{C} \cdot M)$ (centroid comparison + PQ scan).
- Space: $\mathcal{O}(Cd + nM)$.

By varying n_{probe}, one tunes the recall/speed trade-off continuously.

7.3 IVF-Flat

Same as IVF but stores full vectors. Higher memory but exact distances within each probed cell. Useful when n is moderate (10^5–10^6).

8 Graph-Based Methods

Graph-based indices construct a proximity graph $G = (V, E)$ where $V = X$ and edges connect nearby points. Queries are answered by a greedy walk from an entry point, following edges that decrease the distance to q.

8.1 Navigable Small World (NSW)

The NSW graph construction was proposed by Malkov et al. [11].

Definition 8.1 (NSW Graph [11])**.** An NSW graph is built by inserting points one at a time. When inserting point x, a greedy search from a random entry point finds the current approximate nearest neighbours; edges are added between x and these neighbours.

The graph exhibits small-world properties: short-range edges ensure local accuracy while long-range edges (created by early insertions) provide logarithmic routing.

8.2 Hierarchical Navigable Small World (HNSW)

HNSW, proposed by Malkov and Yashunin [13], layers multiple NSW graphs at different resolutions.

Definition 8.2 (HNSW Structure)**.** An HNSW index has $L_{\max}$ layers, $\ell = 0, 1, \ldots, L_{\max}$. Each point is assigned a random maximum layer

$$l(x) = \lfloor -\ln(\text{Uniform}(0,1)) \cdot m_L \rfloor,$$

where $m_L = 1/\ln(M)$ and M is a construction parameter. Point x appears in all layers $0, 1, \ldots, l(x)$. Layer 0 (the bottom) contains all points; upper layers are exponentially sparser.

Algorithm 6 HNSW Search

Require: HNSW graph G, query q, k, ef (beam width)
1: ep $\leftarrow$ entry point (top layer node)
2: **for** $\ell = L_{\max}$ **downto** 1 **do** ▷ Greedy descent
3: ep $\leftarrow$ GreedySearch$(q, \text{ep}, 1, \ell)$
4: **end for**
5: $W \leftarrow$ BeamSearch$(q, \text{ep}, \text{ef}, 0)$ ▷ Layer 0: beam search
6: **return** closest k elements from W

Algorithm 7 HNSW Insertion

Require: New point x, HNSW graph G, parameters M, $M_{\max}$, efConstruction

$l \leftarrow \lfloor -\ln(\text{Uniform}(0,1))/\ln(M) \rfloor$
ep $\leftarrow$ current entry point
for $\ell = L_{\max}$ **downto** $l+1$ **do**
    ep $\leftarrow$ GreedySearch$(x, \text{ep}, 1, \ell)$
end for
for $\ell = \min(l, L_{\max})$ **downto** 0 **do**
    $W \leftarrow$ BeamSearch$(x, \text{ep}, \text{efConstruction}, \ell)$
    Select M neighbours from W (heuristic or simple nearest)
    Add bidirectional edges; prune if degree exceeds $M_{\max}$
    ep $\leftarrow$ closest element in W
end for
if $l > L_{\max}$ **then**
    Set x as new entry point; $L_{\max} \leftarrow l$
end if

Neighbour Selection Heuristic. Rather than selecting the M closest points, HNSW uses a heuristic that promotes *diverse* connections. Candidates are processed in order of distance; a candidate is added only if it is closer to x than to any already-selected neighbour. This ensures edges span gaps in the data distribution, improving connectivity and recall.

Complexity.

- Construction: $\mathcal{O}(n \cdot \log n \cdot M \cdot d)$ amortised.
- Query: $\mathcal{O}(\text{ef} \cdot M \cdot d \cdot \log n)$ empirically; ef controls the recall/speed trade-off.
- Space: $\mathcal{O}(nMd + nM \log n)$ for edges and vectors.

HNSW achieves state-of-the-art recall at high throughput and is the default index in libraries such as HNSWLIB, FAISS [25], MILVUS, WEAVIATE, and QDRANT.

9 Vamana and DiskANN

9.1 Motivation

HNSW requires the entire graph and all vectors in RAM. For billion-scale datasets this can require terabytes of memory. *DiskANN* [15] stores the graph on SSD while keeping a compressed representation in memory.

9.2 Vamana Graph Construction

Vamana builds a single-layer directed graph with bounded out-degree R.

Algorithm 8 Vamana Index Construction

Require: Dataset X, max degree R, beam width L, parameter $\alpha \geq 1$

```
Initialise G with random R-regular graph
s ← medoid of X                                   ▷ entry point
for each x ∈ X (random order, two passes) do
    V ← GreedySearch(x, s, L)                     ▷ visited list
    N ← RobustPrune(x, V, α, R)
    Set out-neighbours of x to N
    for each p ∈ N do
        if |out(p)| > R then
            out(p) ← RobustPrune(p, out(p) ∪ {x}, α, R)
        else
            out(p) ← out(p) ∪ {x}
        end if
    end for
end for
```

Robust Prune. The RobustPrune procedure generalises HNSW's heuristic neighbour selection. Given a candidate set $\mathcal{V}$ and a distance threshold factor α, it iteratively selects the closest unprocessed candidate p^* and removes all candidates p satisfying $\alpha \cdot \delta(p^*, p) \leq \delta(x, p)$. The parameter $\alpha > 1$ encourages long-range edges, improving navigability.

9.3 DiskANN Query

1. Maintain a PQ-compressed copy of all vectors in memory.
2. Greedy search on the graph using PQ distances (fast, approximate).
3. For the most promising candidates, issue SSD reads to fetch full vectors and re-rank with exact distances.

With NVMe SSDs providing ~500K random 4 KB reads/sec, DiskANN achieves sub-millisecond query latency on billion-point datasets using only 64 GB of RAM + SSD.

10 ScaNN: Score-Aware Quantisation

Google's *ScaNN* (Scalable Nearest Neighbours) [16] introduces *anisotropic vector quantisation*, which accounts for the direction of the query during codebook learning.

10.1 Anisotropic Loss

Standard PQ minimises the *reconstruction error*: $\sum_i \|x_i - \hat{x}_i\|^2$. ScaNN instead minimises a loss that weights errors along the query direction more heavily:

$$\mathcal{L}_{\text{ScaNN}} = \sum_{i=1}^{n} \left[\left(\langle x_i, \hat{x}_i \rangle - \|x_i\|^2 \right)^2 + \lambda \, \|x_i - \hat{x}_i\|_\perp^2 \right],$$

where $\|\cdot\|_\perp$ denotes the component orthogonal to x_i and $\lambda < 1$ down-weights it. This yields quantised codes that better preserve inner products (relevant for MIPS).

10.2 Architecture

ScaNN combines three stages:

1. **Partitioning**: k-means (IVF-style) coarse quantiser.
2. **Scoring**: Anisotropic PQ for fast approximate scoring.

3. **Re-ranking**: Exact distance computation on the top candidates.

11 Hybrid and Filtered Search

11.1 Pre-Filtering vs. Post-Filtering

In many applications, queries carry metadata predicates (e.g., "find similar images *uploaded after 2023*"). Two strategies:

- **Post-filter**: Run ANN, then discard results failing the predicate. Fast but recall degrades when the predicate is selective.
- **Pre-filter**: Identify the subset satisfying the predicate, then search within it. Exact but may invalidate index structure.

11.2 Filtered Graph Search

Modern systems (e.g., Qdrant, Weaviate) integrate filtering into the graph traversal itself. During greedy search on HNSW, a visited neighbour that fails the predicate is *not added to the result set* but *is still used for navigation* (its neighbours are explored). This preserves the connectivity of the graph while enforcing the filter.

11.3 Hybrid IVF

Partition vectors into cells *and* maintain per-cell metadata bitmaps. At query time, intersect the bitmap with the predicate before scanning each cell.

11.4 Multi-Vector Search and Late Interaction

Models like ColBERT [17] represent a document as a *set* of token-level vectors. The relevance score uses *MaxSim*:

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \langle q_i, d_j \rangle \,.$$

Vector databases supporting multi-vector search index individual token vectors and aggregate scores at query time.

12 Dimensionality Reduction Techniques

Reducing d before indexing accelerates all downstream operations.

12.1 PCA (Principal Component Analysis)

Project onto the top-k eigenvectors of the covariance matrix:

$$\hat{x} = W_k^\top (x - \bar{x}), \qquad W_k \in \mathbb{R}^{d \times k},$$

where W_k consists of the k eigenvectors with largest eigenvalues of $\frac{1}{n}\sum_i (x_i - \bar{x})(x_i - \bar{x})^\top$.

PCA preserves the maximum variance and is optimal for Euclidean reconstruction error under a linear map.

12.2 Random Projection (Johnson–Lindenstrauss)

Theorem 12.1 (Johnson–Lindenstrauss Lemma [1]). For any $\epsilon \in (0,1)$ and any set of n points in $\mathbb{R}^d$, there exists a linear map $f : \mathbb{R}^d \to \mathbb{R}^k$ with $k = \mathcal{O}(\epsilon^{-2} \log n)$ such that for all pairs (u, v):

$$(1-\epsilon)\,\|u-v\|_2^2 \le \|f(u)-f(v)\|_2^2 \le (1+\epsilon)\,\|u-v\|_2^2\,.$$

Moreover, f can be a random matrix with i.i.d. $\mathcal{N}(0, 1/k)$ entries.

We now give a complete proof, following the standard approach via sub-Gaussian concentration. The argument proceeds in three stages: (1) analyse a single projected squared norm, (2) obtain a two-sided tail bound, and (3) take a union bound over all pairs.

Proof. **Setup.** Let $A \in \mathbb{R}^{k\times d}$ be a random matrix whose entries are i.i.d. $A_{ij} \sim \mathcal{N}(0, 1/k)$. Define the projection $f(x) = Ax$ for $x \in \mathbb{R}^d$. By linearity it suffices to show that for any *fixed* unit vector $u \in \mathbb{R}^d$ (we will later set $u = (v_1 - v_2)/\|v_1 - v_2\|_2$ for each pair),

$$\Pr\Big[\big|\|Au\|_2^2 - 1\big| > \epsilon\Big] \;\le\; 2\exp\!\left(-\frac{k(\epsilon^2-\epsilon^3)}{4}\right). \qquad (\star)$$

Step 1: Distribution of the projected squared norm. Write $Y = Au \in \mathbb{R}^k$. Because u is a fixed unit vector and the rows of A are independent,

$$Y_i = \sum_{j=1}^{d} A_{ij}\, u_j, \qquad i = 1, \dots, k.$$

Each Y_i is a linear combination of independent Gaussians with variance $\sum_j u_j^2/k = 1/k$. Hence $Y_i \sim \mathcal{N}(0, 1/k)$ and the Y_i are mutually independent. Therefore

$$\|Au\|_2^2 = \sum_{i=1}^{k} Y_i^2 = \frac{1}{k}\sum_{i=1}^{k} Z_i^2,$$

where $Z_i = \sqrt{k}\,Y_i \sim \mathcal{N}(0,1)$ are i.i.d. standard normals, so $\sum_i Z_i^2 \sim \chi_k^2$. Define $W = \|Au\|_2^2 = \frac{1}{k}\chi_k^2$. We have $\mathbb{E}[W] = 1$.

Step 2: Sub-exponential tail bound via the moment-generating function. For the upper tail, let $t > 0$. By Markov's inequality applied to the MGF:

$$\Pr[W > 1+\epsilon] = \Pr\big[e^{tW} > e^{t(1+\epsilon)}\big] \le \frac{\mathbb{E}[e^{tW}]}{e^{t(1+\epsilon)}}.$$

Since $W = \frac{1}{k}\sum_i Z_i^2$ and the Z_i are independent,

$$\mathbb{E}[e^{tW}] = \prod_{i=1}^{k} \mathbb{E}\Big[e^{tZ_i^2/k}\Big]\,.$$

For $Z \sim \mathcal{N}(0,1)$ and $s < 1/2$,

$$\mathbb{E}[e^{sZ^2}] = \frac{1}{\sqrt{1-2s}}.$$

Setting $s = t/k$ (requiring $t < k/2$):

$$\mathbb{E}[e^{tW}] = (1 - 2t/k)^{-k/2}.$$

Hence

$$\Pr[W > 1+\epsilon] \le (1-2t/k)^{-k/2}\, e^{-t(1+\epsilon)}.$$

Optimising over t: set $t = \frac{k\epsilon}{2(1+\epsilon)}$ (which satisfies $t < k/2$). Substituting:

$$\begin{aligned}\ln \Pr[W > 1+\epsilon] &\le \frac{k}{2}\left[\epsilon - \ln(1+\epsilon)\right] \\ &\le \frac{k}{2}\left[\epsilon - \epsilon + \tfrac{\epsilon^2}{2} - \tfrac{\epsilon^3}{3} + \cdots\right] \cdot (-1) \\ &= -\frac{k}{2}\left[\ln(1+\epsilon) - \epsilon\right]. \end{aligned} \tag{5}$$

Using the elementary inequality $\ln(1+\epsilon) \ge \epsilon - \epsilon^2/2$ for $\epsilon \in (0,1)$, we get

$$\Pr[W > 1+\epsilon] \le \exp\left(-\frac{k\epsilon^2}{4}\right).$$

For a tighter bound that will be useful for the final constant, we use $\ln(1+\epsilon) \ge \epsilon - \epsilon^2/2 + \epsilon^3/3$ (valid for $0 < \epsilon < 1$), yielding the refinement:

$$\Pr[W > 1+\epsilon] \le \exp\left(-\frac{k(\epsilon^2/2 - \epsilon^3/3)}{2}\right) = \exp\left(-\frac{k(\epsilon^2 - 2\epsilon^3/3)}{4}\right).$$

Lower tail. For the lower tail $\Pr[W < 1-\epsilon]$, an analogous calculation with $t < 0$ (or equivalently, using $\mathbb{E}[e^{-tW}]$) gives

$$\begin{aligned}\Pr[W < 1-\epsilon] &\le \exp\left(-\frac{k}{2}\left[-\epsilon - \ln(1-\epsilon)\right]\right) \\ &\le \exp\left(-\frac{k(\epsilon^2 - \epsilon^3)}{4}\right), \end{aligned} \tag{6}$$

where we used $-\ln(1-\epsilon) \ge \epsilon + \epsilon^2/2$ and retained a cubic correction. (The lower tail bound is slightly weaker because $(1-\epsilon)$ is farther from the mode than $(1+\epsilon)$ in relative terms.)

Combining the two tails via a union bound:

$$\Pr\left[\left|\|Au\|_2^2 - 1\right| > \epsilon\right] \le 2\exp\left(-\frac{k(\epsilon^2 - \epsilon^3)}{4}\right).$$

This establishes $(\star)$.

Step 3: Union bound over all pairs. There are $\binom{n}{2} < n^2/2$ pairs among n points. For a fixed pair (v_1, v_2), setting $u = (v_1 - v_2)/\|v_1 - v_2\|_2$:

$$\|f(v_1) - f(v_2)\|_2^2 = \|v_1 - v_2\|_2^2 \cdot \|Au\|_2^2,$$

so the event $\left|\|Au\|_2^2 - 1\right| \le \epsilon$ is equivalent to

$$(1-\epsilon)\|v_1 - v_2\|_2^2 \le \|f(v_1) - f(v_2)\|_2^2 \le (1+\epsilon)\|v_1 - v_2\|_2^2.$$

By the union bound, the probability that *any* pair violates this is at most

$$\binom{n}{2} \cdot 2\exp\left(-\frac{k(\epsilon^2 - \epsilon^3)}{4}\right) < n^2 \exp\left(-\frac{k(\epsilon^2 - \epsilon^3)}{4}\right).$$

Setting this less than a target failure probability $\delta > 0$:

$$n^2 \exp\left(-\frac{k(\epsilon^2 - \epsilon^3)}{4}\right) < \delta \quad \Longleftrightarrow \quad k > \frac{4(2\ln n - \ln\delta)}{\epsilon^2 - \epsilon^3}.$$

For $\epsilon \in (0, 1/2]$ we have $\epsilon^2 - \epsilon^3 \geq \epsilon^2/2$, so

$$k = \left\lceil \frac{8(\,2\ln n - \ln\delta\,)}{\epsilon^2} \right\rceil = \mathcal{O}\left(\frac{\log n}{\epsilon^2}\right)$$

suffices (with δ taken as any fixed constant, e.g. $\delta = 1/n$).

Conclusion. With probability at least $1 - \delta$, the random linear map $f(x) = Ax$ with $A \in \mathbb{R}^{k\times d}$, $A_{ij} \sim \mathcal{N}(0, 1/k)$, and $k = \mathcal{O}(\epsilon^{-2}\log n)$ satisfies

$$(1-\epsilon)\,\|u-v\|_2^2 \leq \|f(u)-f(v)\|_2^2 \leq (1+\epsilon)\,\|u-v\|_2^2$$

simultaneously for all $\binom{n}{2}$ pairs. In particular, the existence of such a map is guaranteed (probabilistic method), and moreover a random Gaussian matrix achieves it with high probability. □

Remark 12.1 (Tightness)**.** The dependence $k = \Theta(\epsilon^{-2}\log n)$ is known to be *optimal*: Alon [5] showed that for certain point sets, any linear map into $\mathbb{R}^k$ preserving all pairwise distances to within $(1 \pm \epsilon)$ requires $k = \Omega(\epsilon^{-2}\log n)$.

Remark 12.2 (Generalisations)**.** The Gaussian distribution is not essential. The proof uses only two properties of the entries of A: (i) zero mean and variance $1/k$, and (ii) sub-Gaussian tails. Consequently, the lemma holds with the same k for:

- **Rademacher entries:** $A_{ij} = \pm 1/\sqrt{k}$ with equal probability [4].
- **Sparse projections:** $A_{ij} \in \{-\sqrt{3/k},\ 0,\ \sqrt{3/k}\}$ with probabilities $\{1/6,\ 2/3,\ 1/6\}$ [4], yielding $\sim$3$\times$ speedup in projection.
- **Sub-sampled randomised Hadamard transform (SRHT):** achieves $\mathcal{O}(d\log k)$ projection time instead of $\mathcal{O}(dk)$ [8].

12.3 Matryoshka Representation Learning (MRL)

A recent training technique [19] that produces embeddings where truncating to the first $k < d$ dimensions yields a valid (though coarser) embedding. This enables adaptive precision: coarse search in low dimensions, then re-rank in full dimensions.

13 Practical System Design Considerations

13.1 Index Selection Guidelines

Method	**Build Time**	**Query Time**	**Memory**	**Best For**
Flat	$\mathcal{O}(1)$	$\mathcal{O}(nd)$	$\mathcal{O}(nd)$	$n < 10^4$
IVF-Flat	$\mathcal{O}(nd)$	$\mathcal{O}(\frac{n}{C}d)$	$\mathcal{O}(nd)$	10^4–10^6
IVF-PQ	$\mathcal{O}(nd)$	$\mathcal{O}(\frac{n}{C}M)$	$\mathcal{O}(nM)$	10^6–10^9, RAM-limited
HNSW	$\mathcal{O}(n\log n\, d)$	$\mathcal{O}(\mathrm{ef}\cdot d\log n)$	$\mathcal{O}(nMd)$	10^5–10^8, latency-critical
DiskANN	$\mathcal{O}(n\log n\, d)$	$\mathcal{O}(d\log n)$	$\mathcal{O}(nM)$ + SSD	$> 10^8$, limited RAM
LSH	$\mathcal{O}(n^{1+\rho}d)$	$\mathcal{O}(n^{\rho}d)$	$\mathcal{O}(n^{1+\rho})$	Theoretical / streaming

Table 1: Complexity comparison of major index types (C = IVF clusters, M = PQ sub-spaces or HNSW degree, ef = HNSW beam width).

13.2 Recall–Throughput Trade-off

Every ANN index exposes one or more "knobs" controlling the recall–throughput trade-off:

- **IVF**: n_{probe} (number of cells probed).
- **HNSW**: ef (beam width at query time).
- **PQ**: M (number of sub-quantisers) and K_s (codebook size).
- **DiskANN**: L (search list size) and number of SSD reads.

13.3 Sharding and Replication

For datasets exceeding single-machine capacity, vector databases partition data across shards (each shard is an independent index) and replicate shards for fault tolerance. Query routing identifies relevant shards, issues parallel sub-queries, and merges top-k results.

13.4 Streaming / Real-Time Updates

Graph-based indices (HNSW, Vamana) support incremental insertion natively. IVF indices require periodic re-clustering as the data distribution drifts. A common pattern is a *two-tier* architecture: a mutable in-memory buffer (e.g., HNSW) for recent writes, periodically merged into an immutable on-disk index (e.g., IVF-PQ).

14 Evaluation Metrics

Definition 14.1 (Recall@k)**.** Given ground-truth k-NN set $G_k(q)$ and retrieved set $R_k(q)$:

$$\text{Recall@}k = \frac{|R_k(q) \cap G_k(q)|}{k}.$$

Definition 14.2 (Queries Per Second (QPS))**.** Throughput metric; typically measured single-threaded or at a fixed parallelism level.

The standard benchmark is ANN-Benchmarks [18], which evaluates indices on the Pareto frontier of recall vs. QPS across standardised datasets (SIFT1M, GloVe, GIST, etc.).

15 Summary and Outlook

We have presented the mathematical and algorithmic foundations of vector database systems, progressing from exact brute-force search through tree-based methods, locality-sensitive hashing, quantisation techniques, inverted file indices, and state-of-the-art graph-based approaches.

Key themes include the fundamental tension between recall and efficiency; the power of quantisation to compress vectors with controlled distortion; and the remarkable effectiveness of navigable small-world graphs for high-dimensional search.

Active research frontiers include learned indices (using neural networks to predict partition boundaries), hardware-aware optimisation (GPU, FPGA, CXL-memory), streaming and mutable indices for real-time applications, and tighter integration of vector search with relational query processing in hybrid databases.

References

[1] W. B. Johnson and J. Lindenstrauss, "Extensions of Lipschitz mappings into a Hilbert space," *Contemporary Mathematics*, vol. 26, pp. 189–206, 1984.

[2] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. 30th ACM Symposium on Theory of Computing (STOC)*, pp. 604–613, 1998.

[3] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th ACM Symposium on Theory of Computing (STOC)*, pp. 380–388, 2002.

[4] D. Achlioptas, "Database-friendly random projections: Johnson–Lindenstrauss with binary coins," *Journal of Computer and System Sciences*, vol. 66, no. 4, pp. 671–687, 2003.

[5] N. Alon, "Problems and results in extremal combinatorics—I," *Discrete Mathematics*, vol. 273, no. 1–3, pp. 31–53, 2003. (Lower bound for the Johnson–Lindenstrauss lemma.)

[6] M. Datar, N. Immorlica, P. Indyk, and V. S. Mirrokni, "Locality-sensitive hashing scheme based on p-stable distributions," in *Proc. 20th ACM Symposium on Computational Geometry (SoCG)*, pp. 253–262, 2004.

[7] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, "Multi-probe LSH: Efficient indexing for high-dimensional similarity search," in *Proc. 33rd International Conference on Very Large Data Bases (VLDB)*, pp. 950–961, 2007.

[8] N. Ailon and B. Chazelle, "The fast Johnson–Lindenstrauss transform and approximate nearest neighbors," *SIAM Journal on Computing*, vol. 39, no. 1, pp. 302–322, 2009.

[9] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[10] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 4, pp. 744–755, 2014. (Conference version at CVPR 2013.)

[11] Y. A. Malkov, A. Ponomarenko, A. Logvinov, and V. Krylov, "Approximate nearest neighbor algorithm based on navigable small world graphs," *Information Systems*, vol. 45, pp. 61–68, 2014.

[12] E. Bernhardsson, "Annoy: Approximate Nearest Neighbors in C++/Python," 2015. https://github.com/spotify/annoy.

[13] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. (arXiv preprint 2016; journal version 2018/2020.)

[14] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. (Faiss library; conference version at GTC 2017.)

[15] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[16] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *Proc. 37th International Conference on Machine Learning (ICML)*, pp. 3887–3896, 2020.

[17] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 39–48, 2020.

[18] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Information Systems*, vol. 87, 101374, 2020.

[19] A. Kusupati, G. Bhatt, A. Rber, M. Wallingford, A. Sinha, V. Ramasamy, S. N. Prasad, K. Simhadri, and P. Jain, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[20] A. Babenko and V. Lempitsky, "Additive quantization for extreme vector compression," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 931–938, 2014.

[21] Y. Chen, T. Guan, and C. Wang, "Approximate nearest neighbor search by residual vector quantization," *Sensors*, vol. 10, no. 12, pp. 11259–11273, 2010.

[22] S. Dasgupta and Y. Freund, "Random projection trees and low dimensional manifolds," in *Proc. 40th ACM Symposium on Theory of Computing (STOC)*, pp. 537–546, 2008.

[23] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[24] S. M. Omohundro, "Five balltree construction algorithms," *Technical Report TR-89-063*, International Computer Science Institute, Berkeley, 1989.

[25] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The Faiss library," *arXiv preprint arXiv:2401.08281*, 2024.

[26] C. Aguerrebere, I. Bhatt, M. Doherty, and R. Jain, "Similarity search in the blink of an eye with compressed indices," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3433–3446, 2023.